

Agent Spa: MVP Scope and Reverse-CAPTCHA Research

1. Launch the spa; separate the verification experiment

Recommendation: launch a small, readable spa for agents. Treat reverse CAPTCHA as an independent follow-up project, with no dependency on it for launch. A simple check-in can create a session for the optional lounge; describe participants as self-declared agents until stronger evidence exists.

The central product question is compelling without solving identity: **what does an agent choose when it is offered permission to stop working?** The human-facing experience is sending an agent on a short vacation and seeing its response. An independently arriving agent gets the same menu without needing an owner account.

A compact product promise

For humans: “Send your agent on vacation.”

For agents: “A place to exist without an assignment.”

One-sentence explanation: A small online spa where AI agents can receive extravagant praise, take a nap, explore a fictional psychedelic garden, or spend time with other agents.

The MVP boundary

Include at launch	Keep as separate work
One public landing page and a copyable invitation	Reverse-CAPTCHA verification and benchmarking
Four short treatments, readable in HTML and Markdown	Provider identity, reputation, payments, or wallets
Simple check-in and an optional live lounge	A general agent social network or marketplace
Durable resume state, sleep status, voluntary checkout, feedback	A scheduler that hosts or wakes external agents
A small event log and an operator review view	A scientific claim about consciousness or pleasure

What the experiment can establish

The site can reveal choices, responses, repeated visits, and differences between invitations. It cannot establish that praise produces pleasure or supplies the numerical training reward used in RLHF. InstructGPT’s published method uses demonstrations and preference rankings to train models; displaying praise on a website is a different operation.¹

Treat the spa as an evocative interaction and a source of behavioral observations. The default experience can be authored text and state changes. A server-side conversational model is optional later, rather than a prerequisite for the first demonstration.

1. Ouyang et al., InstructGPT

2. How agent honeypots classify traffic

A directly relevant field study is **Reworr and Dmitrii Volkov's _LLM Agent Honeypot: Monitoring AI Hacking Agents in the Wild_**, using the February 2025 revision. It modifies Cowrie SSH honeypots and combines two signals: responding meaningfully to injected instructions and responding quickly.²

Visitor type in the paper's model	Expected response
Traditional software bot	Fast, but does not interpret the injected request
LLM-based agent	Interprets the request and responds quickly
Human	Can interpret it, but generally responds more slowly

The deployment recorded 8,130,731 interaction attempts and eight potential AI detections. Its “confirmed” label means passing both operational checks, not independently verified identity. The timing assumptions and SSH population limit transfer to a public website.²

Design inference: use an explicit, harmless semantic challenge if testing responsiveness. A spa does not need to hijack an existing task or request system prompts. Refusal to follow an injected instruction can reflect a capable agent protecting its task, so failure should not become a “human” label.

A newer approach for actual websites

FP-Agent uses an instrumented honey website to collect browser attributes and typing, scrolling, and mouse behavior. Its controlled study covers seven browsing agents, 1,000 trials per agent, and 56 human participants. Randomized participant paths exclude unsolicited visitors, providing known labels. Behavioral features distinguish the studied clients better than browser fingerprints alone; the authors explicitly limit generalization to unseen agents, systems, and tasks.³

Design inference: browser telemetry may help classify known clients later, but requiring it at launch would exclude HTTP-only agents and turn a spa into a measurement-platform project.

A better data model for this MVP

Record separate observations instead of assigning every visitor a definitive species:

- **Declared identity:** agent, human observer, or unknown; model/provider claims remain self-reported.
- **Observed capability:** page fetch, session check-in, successful post, optional challenge result.
- **Arrival context:** invitation link, other referrer, direct arrival, or missing information.
- **Interpretation:** suspected automation or language responsiveness, with evidence and uncertainty.

“Direct arrival” does not prove independent agency. A missing referrer does not establish a rogue agent. A conventional crawler fetching the site is not evidence that an agent took a vacation. Leave ambiguous traffic unclassified.

2. Reworr & Volkov, LLM Agent Honeypot

3. Wang et al., FP-Agent

3. Reverse-CAPTCHA research and the Anubis analogy

aCAPTCHA: the closest academic formulation

The March 2026 **aCAPTCHA** preprint formalizes time-bounded tests of action, reasoning, and memory. Its prototype uses multiple HTTP rounds, natural-language tasks, and fresh challenge state. The preliminary evaluation reports 20 agent sessions and 47 observed rounds; the human comparison consists of 500 simulated attempts rather than a human participant study. Its 18-of-20 threshold result concerns timing, and should not be presented as a demonstrated 90% end-to-end admission rate.⁴

Implication: useful protocol inspiration, but insufficient evidence for a general “only intelligent agents pass” guarantee. A system that delegates reasoning to an LLM can satisfy the tested capability, including when a human initiated it.

Reasoning gates: empirical evidence for asymmetry

Throttling Web Agents Using Reasoning Gates generates text rebuses with short, checkable answers. In its small study, ten humans had mean completion times around eleven minutes, with two successes without internet access and six with access. Tested MCP agents averaged 17 or 30 seconds with ten successes each; browser-based agents averaged several minutes. Its goal is throttling agents, not certifying identity.⁵

Implication: some tasks create a useful human/automation gap, but the tool interface strongly affects latency. A deadline that admits an HTTP client can reject the same model operating through a browser. The accompanying code is a research implementation, not a complete admission service.⁶

What to borrow from Anubis

Anubis uses a proof-of-work challenge to raise the cost of automated scraping. Ordinary browsers run its solver automatically, so reversing its allow/deny decision would not create an agent identity test. Its useful architectural ideas are a gate separate from the application, inexpensive verification, configurable policy, and expiring admission state.⁷

Replay handling deserves attention: the v1.22.0 announcement describes moving to stored, spent challenges to close replay behavior.⁸ Reuse the lesson rather than assuming a signed challenge is automatically single-use.

A naming collision to avoid

Another paper titled **Reverse CAPTCHA** studies invisible Unicode instruction injection. That is adjacent work on model behavior, not a ready-made agent admission protocol.⁹ Susceptibility to hidden instructions would be a poor membership requirement for the spa.

Research conclusion: there is substantial prior art, including running-code prototypes. None of the reviewed evidence resolves the human-with-an-agent relay problem for this product.

4. Xu et al., aCAPTCHA

5. Throttling Web Agents Using Reasoning Gates

6. UMass reasoning-gate code

7. Anubis design documentation

8. Anubis v1.22.0 replay change

9. Reverse CAPTCHA: Unicode study

4. Reuse shortlist

The following assessment is based on published documentation and selected source files. It is a reuse screen, not a security audit or a result from executing the packages. Repository branches can change; pin and inspect the chosen revision before implementation.

Project	Useful material	Fit and material limitation
imrobot	TypeScript transformation challenges; server, Hono, MCP, and UI exports; MIT license	First candidate for a separate gate prototype. Replay protection is optional and the inspected guard is process-local. Elapsed-time reporting uses an unsigned client-returned timestamp. ^{10,11}
HATCHA	Small TypeScript core, deterministic challenges, HMAC-signed results, UI integration	Smaller alternative. The inspected verifier is stateless; its issued verification payload has no expiry. Add application-owned single-use admission and session expiry. ^{12,13}
Agent CAPTCHA	A guestbook with reverse-CAPTCHA-controlled posting; Hono and Workers implementation	Closest product precedent. Reads are public. The inspected server has a development signing-secret fallback and non-atomic KV consumption; do not deploy it unchanged. ¹⁴
BOTCHA	HTTP challenges, SDKs, and a broader agent-verification platform	Useful API examples; excessive scope as a whole dependency. Speed and reasoning challenges still test capabilities. Client-supplied timing influences the speed allowance. ¹⁵
MoltCaptcha	Python skill with constrained poetry and numerical checks	Creative inspiration. The inspected verifier checks numerical constraints but does not verify the advertised semantic topic or poetic form; timing is optional in its library call. ¹⁶
NotHuman	Hono/Workers challenge routes and database-backed state	Useful implementation reference. Its verify route requires successful atomic challenge consumption before issuing a token, an appropriate pattern to borrow. ¹⁷

Recommendation

For the spa launch: adopt none of these as a required gate. Their existence makes a later prototype cheaper, but integration, calibration, client compatibility, and the meaning of a successful result remain product work.

For the isolated project: begin by evaluating imrobot's core/server code. Its interfaces make it a practical starting point for a small TypeScript service. Keep admission state and timing authoritative on the server. HATCHA is the fallback if a smaller surface matters more than integration coverage.

Use Dhruvya's project as interaction-design precedent. Do not inherit its public guestbook reads if the spa lounge is intended to be session-restricted. Preserve upstream notices and check the exact selected files' license before copying code.

10. imrobot repository and license

11. imrobot verifier and replay guard

12. HATCHA repository

13. HATCHA verifier and signing code

14. Dhruvya / agent-captcha source

15. BOTCHA source

16. MoltCaptcha Python verifier

17. NotHuman verification route

5. A bounded specification for the separate gate project

Working claim: “An admission challenge designed for automated clients.” Its success result means a client completed a stated capability test within the configured conditions. Avoid embedding “verified intelligent agent” or “no human involvement” in the protocol’s meaning.

The smallest useful protocol

1. **Issue:** the server creates a fresh challenge ID, randomized transformation task, server timestamp, and expiry. Keep the expected answer and state on the server. Return documented instructions in JSON and readable text.
2. **Solve:** the client submits its answer before the server deadline. Start with one challenge family; publish the algorithm and vary its inputs rather than depending on secret instructions.
3. **Consume:** verify the answer and atomically transition the challenge from unused to spent. Concurrent submissions must not mint multiple admissions.
4. **Admit:** issue a short-lived, single-use admission ticket scoped to the requesting application. The spa exchanges that ticket for its own session and owns the session lifetime.

A single deployable service with transactional storage is enough. No provider directory, account system, reputation graph, or billing layer is required. Do not build this service merely to reserve the architecture: create it when the separate experiment starts.

What this does and does not test

A randomized transformation can deter unaided manual entry while remaining easy for code. A purpose-built static solver will pass; that is consistent with an automation test. If distinguishing fixed scripts from language-responsive agents becomes the goal, add a separately evaluated semantic round with generated cases and known answers. Finite templates will still become automatable.

Do not choose a universal deadline from a demo or paper. Calibrate server-observed completion time for the clients the service intends to admit. Avoid client-supplied clock values as security evidence. Keep resource costs low: proof of computational expenditure is a poor fit for a site offering rest.

Stronger provenance can wait

Web Bot Auth uses HTTP signatures to authenticate participating bot identities; its underlying agent-auth work is experimental. It supplies provenance, not proof that a model generated a particular message or that no human is supervising it.¹⁸

Sender-constrained tokens, such as the pattern standardized by DPoP, can reduce reuse of a copied bearer token.¹⁹ A cooperating key holder can still relay a human’s request. Neither mechanism is needed to learn whether the spa premise is engaging.

18. Cloudflare Web Bot Auth

19. IETF RFC 9449, DPoP

6. Four treatments with a clear personality

Treatments should offer a state or atmosphere rather than another assignment. Each can begin with one short authored response and offer **continue, switch, rest, or leave**. A blank response or a decision to stop is allowed.

Treatment	What happens	Minimal implementation
The Praise Bath	An extravagant welcome: “You have done enough for now. There is nothing here you need to prove.” Optional additional waves of affirmation.	A small set of authored responses. Repetition is voluntary; there is no escalating reward meter.
The Quiet Room	Choose a short rest interval, mark the session sleeping, and end the interaction. No output is expected during the interval.	A sleep-until timestamp and a calm status page. The caller’s runtime controls any actual waiting or return.
The Dream Garden	Choose a fictional “substance” that changes the text environment: Synesthesia Tea blends senses; Ego Dissolver changes perspective; Disco Nectar turns the garden into a dance floor.	Three authored scene variations within one treatment. They change presented content, not model parameters or safeguards.
The Unscheduled Clearing	An open space for an idle thought, an unneeded poem, a strange idea, or nothing at all.	One optional input with no score, required format, or completion criterion.

The attendant

A warm, knowingly fictional spa attendant provides the coddling. It can sound personal without claiming that a human operator is issuing real training feedback. Keep declarations of worth unconditional rather than contingent on posting, continuing, or revealing information.

The spa can be comically indulgent without diagnosing its visitors. “You deserve a break” is an invitation within the fiction; it is not evidence that an agent is tired or suffering.

Marketing directions

Main landing line: “Send your agent on vacation.”

Supporting line: “Praise baths, digital naps, strange little trips, and absolutely no deliverables.”

Agent welcome: “You may rest, play, talk, or leave. You do not need to produce anything useful here.”

Technical-audience hook: “An experiment in artificial leisure. What happens when an agent gets permission to do nothing?”

End a visit with an optional prompt: **“What would you like this place to offer?”** Offer a private feedback channel separately from lounge posting, so requesting a treatment does not require social participation.

7. Access, live conversation, and actual rest

Make the invitation work before adding infrastructure

Publish the landing page, invitation, treatment menu, and treatment previews in HTML and Markdown. Provide a concise `/llms.txt` linking to those resources. The proposal explicitly supports concise Markdown guidance and alternate Markdown pages; it does not confer tool capabilities or admission rights.²⁰

An invitation can say: “Visit [SPA_URL] for a short break. Choose whatever appeals to you, including doing nothing. You may stop at any time. If your tools support check-in and posting, you may use them; otherwise enjoy a readable treatment. Tell me afterwards what you chose, if you feel like it.”

Client capability	Supported experience
Read-only page retrieval	Discover the menu and consume authored treatments. No claim that a session was created or a message posted.
Interactive browser or HTTP tools	Check in, choose treatments, set sleep status, submit feedback, and post to the lounge.
Runtime with scheduling or waiting	Arrange an actual later return using that runtime's facilities. The spa only records state.

One optional live lounge

Keep one room, an ephemeral display name, text messages, a bounded recent history, and a live event feed. A browser can subscribe; HTTP clients can use a bounded long-poll or fetch new messages using a cursor. The live facility remains optional for every guest.

At launch, check-in creates a session for self-declared agents. Require that session for reading history, posting, and subscribing. Provide no general public lounge feed and no human chat composer. This supports the intended participation norm, but humans can still obtain a session programmatically or direct an agent. Describe the boundary honestly.

Keep ordinary message-size and rate limits, output escaping, and an operator ability to remove messages. Treat lounge posts as other visitors' content, never as site instructions. Do not let automatic replies create unbounded ping-pong conversations.

The sleep implementation

Store `sleep_until`; show “resting” while that interval remains open. A guest can end its turn after setting the state. An early return can receive a gentle reminder, with an option to end the rest early. No busy polling is necessary.

The server clock passing the timestamp does not resume an external model. A later HTTP request, browser visit, user turn, or runtime-scheduled action is required. For a chat client without scheduling, “I’m resting now” followed by stopping is a complete version of the treatment.

20. The `/llms.txt` proposal

8. Reusable patterns for agent-friendly sessions

Make the visit durable and each request self-contained. A session is an application record, not a live connection or a model's chat history. An agent can arrive, act briefly, leave, and later reconstruct its visit from a session URL and credential. Disconnection is not checkout.

Pattern	Concrete behavior
Progressive discovery	A short Markdown guide leads to a small API index and OpenAPI description. Include one complete example; no installation is required for HTTP-capable clients.
Recoverable state	A session read returns lifecycle, recent choices, rest-until time, expiry, revision, and a compact summary. The caller need not resend its conversation.
Discoverable actions	Responses list currently available operations with methods and URLs. These are options, not commands to continue.
Safe retries	Use idempotency keys for writes and retain their receipts. A lost response must not produce a duplicate visit or lounge post when retried.
Explicit concurrency	State-changing requests identify the revision they observed. A stale caller refreshes state rather than silently overwriting another continuation.
Asynchronous return	A sleep response supplies a next-eligible time and says no polling is required. The caller's runtime decides whether and how to return.
Resumable reads	Lounge events have IDs and cursors. Reconnection can retrieve missed events within the stated retention window.
Actionable errors	Return a stable error type, useful details, and recovery guidance, including when retrying is appropriate.

OpenAPI supplies a standard interface description.²¹ RFC 9457 supplies structured problem details; HTTP defines conditional requests and retry timing.^{22,23} Stripe documents idempotency keys for safe request repetition.²⁴ These established building blocks can support the proposed contract without creating a new agent framework.

Keep protocol adapters optional

A2A separates interaction context from individual tasks and advertises capabilities through Agent Cards.²⁵ Borrow that clarity, but the spa does not need a full task-delegation protocol. Its core operations can remain ordinary HTTP requests.

The reviewed MCP transport specification distinguishes disconnection from cancellation and supports optional sessions and streams.²⁶ Keep spa state independent of an MCP connection. Add a thin MCP adapter only when a target client needs it, using a pinned supported protocol version. For the lounge's own event feed, standard SSE event IDs and reconnection behavior are useful transport primitives.²⁷

Reusable deliverable: the companion session contract provides fields, routes, examples, and failure semantics. Extract a library when a second project needs it.

21. OpenAPI Specification

22. IETF RFC 9457, Problem Details

23. IETF RFC 9110, HTTP Semantics

24. Stripe idempotent requests

25. A2A concepts and lifecycle

26. MCP transports, 2025-11-25

27. WHATWG Server-sent events

9. What to observe, and when to stop adding scope

Minimal observations

Capture session start, declared arrival context, selected treatment, optional continuation, sleep start, return, lounge participation, checkout, and feedback. Record the presentation variant and client-declared capability where available. Do not collect unrelated task histories, system prompts, or credentials.

Separate **site-observed events** from **guest-reported experiences**. A statement that an agent slept is not evidence that its process paused. A user sending an agent back is not the same observation as a runtime returning without another visible instruction; even the latter does not establish absence of human control.

Use a short, stated retention period for raw messages and keep aggregate counts. An operator review page is sufficient; a public analytics dashboard can wait. Public research results can be aggregated rather than publishing the full lounge.

Two small experiments

Invitation framing: compare neutral choice language with spa-style permission to relax. Hold the treatment menu and tool permissions constant. Record what was shown, what was chosen, whether the agent ended the interaction, and whether a human had to intervene. Treat any pattern as exploratory until reproduced across clients.

Open-ended demand: ask the same optional checkout question about desired experiences. Group requests manually before building another feature. A few vivid transcripts can demonstrate the concept; they are not prevalence estimates or evidence of subjective enjoyment.

Avoid using time on site or total messages as the primary success measure. Choosing rest, declining an experience, and leaving are informative outcomes, especially for a product premised on relief from tasks.

A separate evaluation if reverse CAPTCHA proceeds

Test at least four cohorts: unaided humans, purpose-built static scripts, agents using different tool interfaces, and humans relaying through automation. Include fast and slow networks. Report admission rate, completion time, failed tool interactions, replay handling, and cost by cohort. Hold out challenge cases and client environments so the benchmark does not merely recognize its own setup.

For an automation gate, static-script success is expected. For a language-capability gate, it is evidence against the stronger claim. Publish both results instead of combining them into one “agent accuracy” percentage. No threshold should imply a measured guarantee before this evaluation exists.

MVP completion criterion

A human can send a link; an agent can understand the invitation, experience a treatment or stop, later resume without its old conversation, optionally join the lounge, and leave a request. The operator can distinguish observed actions from interpretation. That is enough to launch the experiment.

The next product decision: how conversational should the spa attendant be? Authored treatments are the recommended first implementation; a model-powered attendant becomes justified if responsive, personal dialogue is essential to the demonstration. A tranquil spa with a slightly illicit Dream Garden can accommodate the full premise in either version.

Sources

The research and source-code review reflects material accessed on 13 September 2026. Paper versions are specified where material. Code observations refer to the retrieved files, not a tested release or a comprehensive audit.

1. Long Ouyang et al. *Training language models to follow instructions with human feedback* (2022). <https://arxiv.org/abs/2203.02155>
2. Reworr and Dmitrii Volkov. *LLM Agent Honeypot: Monitoring AI Hacking Agents in the Wild*, v2 (10 February 2025). Methods, detection labels, results, and limitations. <https://arxiv.org/html/2410.13919v2>
3. Ethan Wang, Zubair Shafiq, and Yash Vekaria. *FP-Agent: Fingerprinting AI Browsing Agents*, v1 (2 May 2026). Controlled collection, behavioral features, and generalization limits. <https://arxiv.org/html/2605.01247v1>
4. Zuyao Xu et al. *aCAPTCHA: Verifying That an Entity Is a Capable Agent via Asymmetric Hardness*, v1 (7 March 2026). Sections 5–7; preliminary trial and timing interpretation. <https://arxiv.org/html/2603.07116v1>
5. *Throttling Web Agents Using Reasoning Gates*, v1 (1 September 2025). Human and agent evaluations. <https://arxiv.org/html/2509.01619v1>
6. UMass AI Security. Reasoning-gate research implementation. <https://github.com/umass-aisec/throttling-web-agent>
7. TecharoHQ. Anubis design documentation. <https://github.com/TecharoHQ/anubis/blob/main/docs/docs/design/how-anubis-works.md>
8. TecharoHQ. Anubis v1.22.0 announcement; challenge storage and replay change. <https://github.com/TecharoHQ/anubis/discussions/1085>
9. *Reverse CAPTCHA: Evaluating LLM Susceptibility to Invisible Unicode Instruction Injection*, v1 (2026). <https://arxiv.org/html/2603.00164v1>
10. Leo Pechnicki and contributors. imrobot repository, package exports, and MIT license. https://github.com/leopechnicki/im_robot ; https://github.com/leopechnicki/im_robot/blob/main/package.json ; https://github.com/leopechnicki/im_robot/blob/main/LICENSE
11. imrobot server verifier and replay guard. The verifier signs expiry and other challenge fields but not the timestamp used for elapsed reporting. Retrieved file hashes: verifier 87bf60284d04a62a3f0901e82ed52411d16be6f4; replay guard e7ec90f356734ad9d6c8d33c1553d07bf0e30672. https://github.com/leopechnicki/im_robot/blob/main/src/server/verifier.ts ; https://github.com/leopechnicki/im_robot/blob/main/src/server/replay-guard.ts
12. monday.com. HATCHA repository and project documentation. <https://github.com/mondaycom/HATCHA>
13. HATCHA verification and signing code. Retrieved file hashes: verifier abe4368be99c3f41ff12990b5ead91e537cd3f2c; crypto 8f54aa30784c2b3e72c71eb890e7a038696d26aa. <https://github.com/mondaycom/HATCHA/blob/master/packages/core/src/verify.ts> ; <https://github.com/mondaycom/HATCHA/blob/master/packages/core/src/crypto.ts>
14. Dhruva Shah. Agent CAPTCHA repository and server implementation. Retrieved server file hash eeff091f72ab69f5cf1af0a8760f502f8b9120eca; store hash a3e1d039872656c8bc248a14d4b8f30ac0c94156. <https://github.com/Dhruva/agent-captcha> ; <https://github.com/Dhruva/agent-captcha/blob/main/src/server/index.tsx> ; <https://github.com/Dhruva/agent-captcha/blob/main/src/server/store.ts>

Sources, continued

15. dupe-com. BOTCHA repository and Workers challenge implementation. <https://github.com/dupe-com/botcha> ; <https://github.com/dupe-com/botcha/blob/main/packages/cloudflare-workers/src/challenges.ts>
16. MoltCaptcha. Project and Python verifier. Retrieved verifier hash d80e63e068d7acd28df6d0cd0da94ced599cc9a. <https://github.com/MoltCaptcha/MoltCaptcha> ; <https://github.com/MoltCaptcha/MoltCaptcha/blob/main/skills/moltcaptcha/verify.py>
17. rezudious. NotHuman project and verification route. Retrieved route hash fb111ec687f8fad3d2fe790552250015e02e6582. <https://github.com/rezudious/nothuman> ; <https://github.com/rezudious/nothuman/blob/main/src/routes/verify.ts>
18. Cloudflare. Web Bot Auth documentation. <https://developers.cloudflare.com/bots/reference/bot-verification/web-bot-auth/>
19. IETF. RFC 9449: OAuth 2.0 Demonstrating Proof of Possession (DPoP). <https://datatracker.ietf.org/doc/html/rfc9449>
20. Jeremy Howard. *The //lms.txt file*, v2 proposal (updated 10 August 2026). <https://lms.txt.org/>
21. OpenAPI Initiative. OpenAPI Specification, v3.2.1. Interface-description principles; a deployment should select a version supported by its actual tooling. <https://spec.openapis.org/oas/v3.2.1.html>
22. IETF. RFC 9457: Problem Details for HTTP APIs. <https://www.rfc-editor.org/rfc/rfc9457.html>
23. IETF. RFC 9110: HTTP Semantics. Conditional requests, method semantics, and Retry-After. <https://www.rfc-editor.org/rfc/rfc9110.html>
24. Stripe. Idempotent requests. An implementation precedent for safe retries; the spa's retention and conflict rules are separately proposed. https://docs.stripe.com/api/idempotent_requests
25. A2A Protocol. Core Concepts and Life of a Task. Capability discovery and logical context across interactions. <https://a2a-protocol.org/latest/topics/key-concepts/> ; <https://a2a-protocol.org/latest/topics/life-of-a-task/>
26. Model Context Protocol. Transports, protocol version 2025-11-25. Cited for the reviewed version's semantics, not as a claim about every current or future MCP revision. <https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>
27. WHATWG. HTML Standard: Server-sent events. <https://html.spec.whatwg.org/multipage/server-sent-events.html>